

SISTEMAS PROGRAMABLES

delay() frente a millis() en Arduino

Reporte de práctica

Integrantes del equipo

Daniel Bustamante

Iván Luan

Materia: Sistemas Programables

Plataforma: Arduino UNO R4 WiFi

1. Introducción

Al programar sistemas embebidos con Arduino, uno de los problemas más frecuentes es medir el tiempo entre eventos sin congelar el resto del programa. Para lograrlo existen dos herramientas muy usadas: la función `delay()` y la función `millis()`. En este reporte se comparan ambas a partir de una práctica de laboratorio en la que se manejaron tres LEDs que simulan un semáforo vehicular.

2. Objetivo

Entender en qué se diferencian `delay()` y `millis()` al controlar la temporización en Arduino, reconociendo las ventajas, las limitaciones y los casos de uso de cada una, mediante la construcción de un circuito con tres LEDs.

3. Marco teórico

3.1 La función `delay()`

`delay(ms)` suspende por completo la ejecución del programa durante los milisegundos que se le indiquen. Mientras dura esa pausa, el microcontrolador no puede leer entradas, actualizar otras salidas ni realizar ninguna otra tarea. Su uso es muy sencillo, pero es una función bloqueante.

3.2 La función `millis()`

`millis()` entrega el número de milisegundos que han pasado desde que se encendió el Arduino. En vez de pausar el programa, se almacena una marca de tiempo de referencia y, en cada vuelta de `loop()`, se compara el tiempo actual con dicha referencia; cuando la diferencia llega al intervalo deseado, se ejecuta la acción correspondiente. Así el programa nunca se detiene y puede atender varias tareas a la vez (funcionamiento no bloqueante).

4. Desarrollo práctico

Se armó el control de tres LEDs (rojo, amarillo y verde), conectados a los pines digitales D2, D3 y D4 respectivamente, para reproducir la secuencia de un semáforo vehicular: verde durante 8 segundos, amarillo durante 2 segundos y rojo durante 6 segundos.

4.1 Conexiones utilizadas

- Rojo → pin D2, con resistencia de 220–330 Ω y cátodo a GND.
- Amarillo → pin D3, con resistencia de 220–330 Ω y cátodo a GND.
- Verde → pin D4, con resistencia de 220–330 Ω y cátodo a GND.

4.2 Versión con `delay()`

El código siguiente enciende un LED tras otro y usa `delay()` para fijar cuánto dura cada color:

```
// Semáforo de 3 LEDs
// Arduino UNO R4 WiFi
// Usa delay()

const int ROJO = 2;
const int AMARILLO = 3;
const int VERDE = 4;
```

```

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  digitalWrite(VERDE, HIGH);
  digitalWrite(AMARILLO, LOW);
  digitalWrite(ROJO, LOW);
  delay(8000);

  digitalWrite(VERDE, LOW);
  digitalWrite(AMARILLO, HIGH);
  digitalWrite(ROJO, LOW);
  delay(2000);

  digitalWrite(VERDE, LOW);
  digitalWrite(AMARILLO, LOW);
  digitalWrite(ROJO, HIGH);
  delay(6000);
}

```

4.3 Versión con millis()

Este código produce exactamente la misma secuencia, pero sin bloquear el programa: una variable de estado (estado) y una marca de tiempo (tiempoAnterior) determinan cuándo pasar al siguiente color.

```

// Semáforo de 3 LEDs usando millis()
// Arduino UNO R4 WiFi

const int ROJO = 2;
const int AMARILLO = 3;
const int VERDE = 4;

const unsigned long TIEMPO_VERDE = 8000;
const unsigned long TIEMPO_AMARILLO = 2000;
const unsigned long TIEMPO_ROJO = 6000;

unsigned long tiempoAnterior = 0;
int estado = 0; // 0=Verde 1=Amarillo 2=Rojo

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
  digitalWrite(VERDE, HIGH);
  tiempoAnterior = millis();
}

void loop() {
  unsigned long tiempoActual = millis();

  if (estado == 0 && tiempoActual - tiempoAnterior >= TIEMPO_VERDE) {
    estado = 1;
    tiempoAnterior = tiempoActual;
    digitalWrite(VERDE, LOW);

```

```

    digitalWrite(AMARILLO, HIGH);
}
else if (estado == 1 && tiempoActual - tiempoAnterior >= TIEMPO_AMARILLO) {
    estado = 2;
    tiempoAnterior = tiempoActual;
    digitalWrite(AMARILLO, LOW);
    digitalWrite(ROJO, HIGH);
}
else if (estado == 2 && tiempoActual - tiempoAnterior >= TIEMPO_ROJO) {
    estado = 0;
    tiempoAnterior = tiempoActual;
    digitalWrite(ROJO, LOW);
    digitalWrite(VERDE, HIGH);
}
}
}

```

5. Análisis comparativo

Al contrastar las dos implementaciones se observaron estas diferencias principales:

Aspecto	delay()	millis()
Bloqueo del programa	Sí, detiene por completo la ejecución mientras dura la espera.	No, el programa sigue corriendo; solo se compara el tiempo transcurrido.
Lectura de sensores y botones	Imposible durante la pausa.	Se puede hacer en cada vuelta de loop(), sin perder eventos.
Varias tareas a la vez	Muy difícil o imposible de manera independiente.	Permite manejar varias temporizaciones al mismo tiempo, cada una por separado.
Precisión a largo plazo	Aceptable para pruebas simples.	Igual de precisa; además, no se descompone por depender de varios delays encadenados.
Complejidad del código	Muy fácil de escribir y de leer.	Exige variables de estado y comparaciones de tiempo.
Uso recomendado	Pruebas rápidas y prototipos que no necesitan reaccionar a eventos.	Proyectos con botones, sensores o varios procesos (máquinas de estados).

6. Conclusiones

Las dos funciones sirven para temporizar eventos, pero se adaptan a situaciones distintas. delay() resulta útil en pruebas rápidas o circuitos muy simples en los que el Arduino no necesita atender otros eventos mientras espera. millis(), por su parte, es la opción indicada para proyectos reales de sistemas programables, porque mantiene el programa activo en todo momento, permite leer botones o sensores sin perder pulsaciones y controla varios procesos de forma independiente, como exigen las máquinas de estados finitos más elaboradas (por ejemplo, un semáforo con solicitud peatonal).

Como equipo, concluimos que dominar millis() es indispensable para avanzar hacia sistemas embebidos más sólidos, pues evita los bloqueos que delay() provoca en aplicaciones interactivas.